
raft-rx — User guide: Python

FSM-based Raft implementation over rxnet

2026-05-10

Contents

raft_rx - Python User Guide	5
Part I - The Library	7
1. Overview	7
2. Architecture	8
3. Key Concepts	8
Roles	8
Log, Terms and Commit	9
Membership Configuration	9
4. Implementing an Application	9
Example: Key-Value Application	10
5. Building a Cluster	11
Running Ticks	12
6. Submitting Commands	12
7. Membership Changes	13
8. Storage Format	13
meta.json	14
log.json	14
snapshot.json	14
9. Observability and Tracing	15
JSONL Telemetry	15
rxnet Trace	15
10. Shell	15
Generic Commands	16
KV Commands	16
11. Standalone HTTP Demo	17
Run	17
CLI Commands	17
REST API	18
12. Public API Reference	19
Package Exports	19

NodeConfig	20
RaftCluster	20
RaftNode	21
13. Building and Testing	21
14. Troubleshooting	22
No leader is elected	22
Commands are not committed	22
A joining node does not participate	22
Snapshot restore misses state	22

List of Figures

raft_rx - Python User Guide

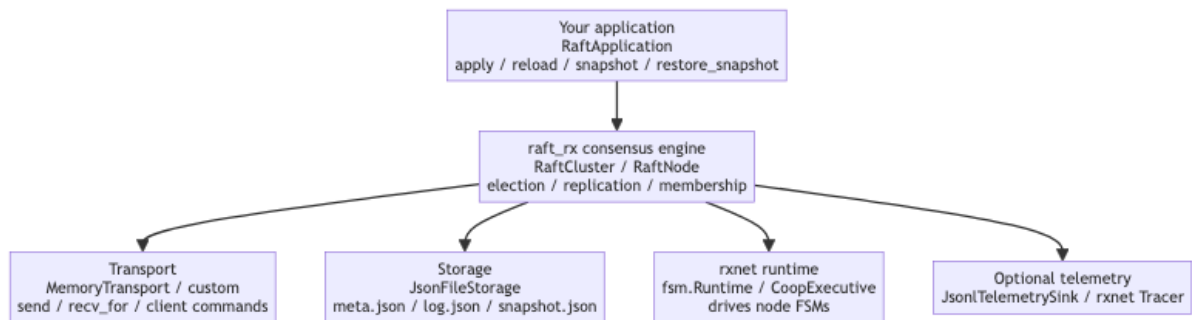
Part I - The Library

1. Overview

`raft_rx` is a Python package that implements the Raft consensus algorithm on top of the `rxnet` FSM runtime. It provides:

- **Replicated state machine** - commands are appended to a distributed log, committed by majority quorum, and applied to your application in order.
- **Leader election** - deterministic tests via `ManualClock`, or wall-clock operation via `SystemClock`.
- **Membership changes** - joint-consensus reconfiguration through `request_membership_change`.
- **Persistent storage** - term, vote, log, snapshot and membership configuration are stored as JSON files.
- **Pluggable transport** - the package includes deterministic in-memory transport; the standalone demo shows an HTTP transport.
- **Pluggable application** - implement a small protocol to attach your own state machine.
- **Optional observability** - JSONL telemetry and `rxnet` trace export can be enabled per cluster.

2. Architecture



Layer	Type	Responsibility
Application	<code>RaftApplication</code>	Your state machine: what commands mean
Consensus engine	<code>RaftCluster</code> / <code>RaftNode</code>	Raft protocol, log replication and membership
Transport	<code>MemoryTransport</code> or compatible object	Deliver Raft messages and client commands
Storage	<code>JsonFileStorage</code>	Durable protocol state and snapshots
Runtime	<code>rxnet.fsm.Runtime</code>	Ticks the Raft FSMs
Telemetry	<code>JsonlTelemetrySink</code> / <code>NullTelemetrySink</code>	Optional structured events

3. Key Concepts

Roles

Every running node is in one of three roles:

Role	Description
<code>FOLLOWER</code>	Accepts log entries and votes in elections
<code>CANDIDATE</code>	Requests votes after its election timeout expires
<code>LEADER</code>	Accepts client commands, replicates and commits them

Log, Terms and Commit

- Each client command becomes a `LogEntry(index, term, command, leader_id)`.
- The leader replicates entries to a quorum, then advances `commit_index`.
- Committed entries are applied through your application's `apply` method.
- `term` is a monotonically increasing election epoch.

Membership Configuration

Membership is represented by `ClusterConfiguration`:

- `old_members`: current stable set, or leaving set during joint consensus.
- `new_members`: joining set during joint consensus, otherwise `None`.
- `index`: log index where the configuration was written.

`mode()` returns `"stable"` or `"joint"`. During joint consensus, commit requires a majority in both `old_members` and `new_members`.

4. Implementing an Application

Your application must implement the `RaftApplication` protocol:

```
1 from raft_rx import Command
2
3
4 class RaftApplication:
5     def apply(self, command: Command) -> None: ...
6     def reload(self) -> None: ...
7     def snapshot(self) -> object: ...
8     def restore_snapshot(self, snapshot: object) -> None: ...
```

Commands are simple dataclasses:

```
1 from raft_rx import Command
2
3 Command(op="set", key="color", value="blue")
4 Command(op="delete", key="color")
```

Method	When it is called	What it should do
<code>apply</code>	Once per committed log entry, in index order	Mutate application state
<code>reload</code>	Restart with no snapshot	Reset or reload the empty/base state
<code>snapshot</code>	Log compaction	Return a JSON-serializable snapshot object
<code>restore_snapshot</code>	Restart from snapshot	Restore state from the snapshot object

Correctness rule: mutate replicated state only from `apply`. Reads can inspect the local applied state, but writes must enter the leader's log.

Example: Key-Value Application

```
1 import json
2 from pathlib import Path
3
4 from raft_rx import Command
5
6
7 class KVApp:
8     def __init__(self, path: Path) -> None:
9         self.path = path
10        self.path.parent.mkdir(parents=True, exist_ok=True)
11        self.data: dict[str, str] = {}
12        self.reload()
13
14    def reload(self) -> None:
15        if self.path.exists():
16            self.data = {
17                str(k): str(v)
18                for k, v in json.loads(self.path.read_text()).items()
19            }
```

```

20         else:
21             self.data = {}
22
23     def apply(self, command: Command) -> None:
24         if command.op == "set":
25             if command.value is None:
26                 raise ValueError("set requires value")
27             self.data[command.key] = command.value
28         elif command.op == "delete":
29             self.data.pop(command.key, None)
30         else:
31             raise ValueError(f"unsupported op: {command.op}")
32         self.path.write_text(json.dumps(self.data, indent=2, sort_keys=
33                                 True))
34
35     def get(self, key: str) -> str | None:
36         return self.data.get(key)
37
38     def snapshot(self) -> object:
39         return dict(self.data)
40
41     def restore_snapshot(self, snapshot: object) -> None:
42         self.data = {str(k): str(v) for k, v in dict(snapshot).items()}
43         self.path.write_text(json.dumps(self.data, indent=2, sort_keys=
44                                 True))

```

The repository version in `python/examples/kv_app.py` also writes via a temporary file and atomic replace.

5. Building a Cluster

Use `RaftCluster` for local deterministic simulations and tests.

```

1  from pathlib import Path
2
3  from raft_rx import ManualClock, NodeConfig, RaftCluster
4  from raft_rx.cluster import ClusterNodePaths
5
6
7  root = Path("var/my-cluster")
8  cluster = RaftCluster(clock=ManualClock())
9  node_ids = ["n1", "n2", "n3"]
10
11  for i, node_id in enumerate(node_ids):
12      peers = [peer for peer in node_ids if peer != node_id]

```

```

13     cluster.add_node(
14         NodeConfig(
15             node_id=node_id,
16             peers=peers,
17             election_timeout_ms=150 + i * 100,
18             heartbeat_interval_ms=50,
19         ),
20         ClusterNodePaths(
21             storage_dir=root / "storage",
22             telemetry_path=root / "telemetry" / f"{node_id}.jsonl",
23         ),
24         application=KVApp(root / "kv" / f"{node_id}.json"),
25     )

```

`NodeConfig.initial_members` is optional. If omitted, the node starts with `[node_id, *peers]`. Use an explicit list when you need all nodes to share a particular initial ordering.

Running Ticks

With `ManualClock`, each tick advances time explicitly:

```

1 cluster.run(40, advance_ms=10)
2 leader = cluster.leader()

```

With `SystemClock`, use an `rxnet` executive, as shown by the standalone demo:

```

1 from rxnet.coop import CoopExecutive
2
3 executive = CoopExecutive()
4 executive.add(cluster.runtime)
5 executive.run()

```

6. Submitting Commands

Commands must reach the leader to be committed.

```

1 from raft_rx import Command
2
3 leader = cluster.leader()
4 if leader is None:
5     raise RuntimeError("no leader")
6
7 leader.submit_command(Command(op="set", key="color", value="blue"))

```

```
8 cluster.run(30, advance_ms=10)
```

`RaftCluster.submit_to_leader` is a convenience wrapper:

```
1 ok = cluster.submit_to_leader(Command(op="delete", key="color"))
```

If a command is submitted to a follower in the core in-memory transport, that follower drains and rejects it. Production-style forwarding belongs in the application or transport layer; the standalone HTTP demo implements that.

7. Membership Changes

Call `request_membership_change` on the current leader with the full target member list, not a delta.

```
1 leader = cluster.leader()
2 if leader is None:
3     raise RuntimeError("no leader")
4
5 cluster.provision_node("n4", members=["n1", "n2", "n3", "n4"], running=
    True)
6 leader.request_membership_change(["n1", "n2", "n3", "n4"])
7 cluster.run(120, advance_ms=10)
```

To remove a node, omit it from the target list:

```
1 leader.request_membership_change(["n1", "n2", "n3"])
```

Internally the membership FSM writes:

1. `cluster.enter_joint`
2. `cluster.leave_joint`

During the joint phase, quorum requires a majority from both old and new configurations.

8. Storage Format

`JsonFileStorage(root, node_id)` stores files under `{root}/{node_id}/`:

```
1 var/my-cluster/storage/└─  
2   n1/└─  
3     meta.json└─  
4     log.json└─  
5     snapshot.json
```

meta.json

Contains protocol metadata:

```
1 {  
2   "commit_index": 2,  
3   "compaction_threshold": 32,  
4   "configuration_index": 0,  
5   "current_term": 1,  
6   "new_members": null,  
7   "old_members": ["n1", "n2", "n3"],  
8   "snapshot_last_included_index": 0,  
9   "snapshot_last_included_term": 0,  
10  "voted_for": "n1"  
11 }
```

log.json

Contains un-compacted log entries:

```
1 [  
2   {  
3     "index": 1,  
4     "term": 1,  
5     "leader_id": "n1",  
6     "command": {"op": "set", "key": "color", "value": "blue"}  
7   }  
8 ]
```

snapshot.json

Contains compaction metadata plus your application snapshot:

```
1 {  
2   "application": {"color": "blue"},  
3   "last_included_index": 32,  
4   "last_included_term": 4  
5 }
```

`JsonFileStorage` writes JSON via temp file plus replace.

9. Observability and Tracing

JSONL Telemetry

Pass a telemetry path when adding each node:

```
1 ClusterNodePaths(  
2     storage_dir=root / "storage",  
3     telemetry_path=root / "telemetry" / f"{node_id}.jsonl",  
4 )
```

Each line is a JSON object with time, role, term, event and payload. The shell command `events [NODE ...]` displays the last events.

rxnet Trace

Enable tracing on the cluster:

```
1 cluster = RaftCluster(clock=ManualClock(), trace_enabled=True)
```

Export the trace:

```
1 cluster.export_trace(root / "trace.bin")  
2 cluster.report_trace(root / "trace.html")
```

The example `python/examples/kv_reconfigure_trace.py` generates:

- `python/var/python-reconfigure-trace/trace.bin`
 - `python/var/python-reconfigure-trace/trace.html`
-

10. Shell

Run the generic shell:

```
1 uv run --project python python/tools/raftsh.py
```


Run the KV shell:

```
1 uv run --project python python/examples/kv_shell.py
```

Generic Commands

Command	Description
<code>status</code>	Cluster status and per-node summary
<code>leader</code>	Current leader
<code>tick [N] [MS]</code>	Advance the simulation
<code>autotick [MS]</code>	Configure background ticking; 0 disables it
<code>stop NODE</code>	Stop a node
<code>start NODE</code>	Start a node
<code>restart NODE</code>	Stop and start a node
<code>members [NODE]</code>	Show stable or joint membership
<code>addnode NODE</code>	Provision and add a node via joint consensus
<code>rmnode NODE</code>	Remove a node via joint consensus
<code>log NODE</code>	Print local log entries and commit status
<code>maxlog [ENTRIES]</code>	Read or update compaction threshold
<code>dump</code>	Print summaries as JSON
<code>trace [PATH]</code>	Export <code>trace.bin</code> if tracing is enabled
<code>trace_report [PATH]</code>	Export an HTML trace report
<code>events [NODE ...]</code>	Show recent JSONL telemetry
<code>quit/exit</code>	Leave the shell

KV Commands

The KV shell registers additional commands:

Command	Description
<code>set KEY VALUE</code>	Submit a replicated write to the leader
<code>get KEY [NODE]</code>	Read local applied state from a node
<code>delete KEY</code>	Submit a replicated delete to the leader

11. Standalone HTTP Demo

`python/examples/demo_app` is a separate application that consumes `raft-rx` as an external dependency. Each process is one Raft node and uses `HOST:PORT` as its node ID.

Run

From `python/`:

```
1 uv run --project examples/demo_app raft-rx-demo --bind 127.0.0.1:7400
2 uv run --project examples/demo_app raft-rx-demo --join 127.0.0.1:7400
  --bind 127.0.0.1:7402
```

Useful flags:

Flag	Description
<code>--bind HOST:PORT</code>	Address to bind and advertise as node ID
<code>--join HOST:PORT</code>	Existing node to join at startup
<code>--data-dir DIR</code>	Persistent state directory
<code>--tick-ms MS</code>	Runtime tick period; default 25

CLI Commands

Command	Description
<code>status</code>	Local node status
<code>members</code>	Membership configuration
<code>log [LIMIT]</code>	Local log entries
<code>set KEY VALUE</code>	Replicated write; forwarded to the leader if needed
<code>get KEY</code>	Local read
<code>delete KEY</code>	Replicated delete
<code>addnode HOST:PORT</code>	Add a node ID/address to membership
<code>rmnode HOST:PORT</code>	Remove a node ID/address from membership
<code>join HOST:PORT</code>	Join the target cluster
<code>merge HOST:PORT</code>	Merge this cluster with another cluster
<code>stop/start</code>	Stop or resume local Raft processing
<code>quit</code>	Stop the process

REST API

Endpoint	Method	Purpose
<code>/status</code>	GET	Local node summary
<code>/cluster/config</code>	GET	Membership configuration
<code>/log?limit=N</code>	GET	Local log entries
<code>/kv/<KEY></code>	GET	Local KV read
<code>/kv/set</code>	POST	Replicated <code>set</code> ; body <code>{"key": "...", "value": "..."} </code>
<code>/kv/delete</code>	POST	Replicated delete; body <code>{"key": "..."} </code>
<code>/raft/message</code>	POST	Incoming Raft protocol message

Endpoint	Method	Purpose
<code>/raft/client-command</code>	POST	Forwarded client command
<code>/cluster/add-node</code>	POST	Add a node; body <code>{"node": "HOST:PORT"}</code>
<code>/cluster/remove-node</code>	POST	Remove a node; body <code>{"node": "HOST:PORT"}</code>
<code>/cluster/join</code>	POST	Add caller node to the cluster
<code>/cluster/membership-request</code>	POST	Request a target member list
<code>/cluster/merge</code>	POST	Merge membership with another cluster
<code>/node/stop</code>	POST	Stop local Raft processing
<code>/node/start</code>	POST	Resume local Raft processing

12. Public API Reference

Package Exports

```
1 from raft_rx import (  
2     ClusterConfiguration,  
3     Command,  
4     JsonFileStorage,  
5     JsonlTelemetrySink,  
6     LogEntry,  
7     ManualClock,  
8     MemoryTransport,  
9     Message,  
10    MessageKind,  
11    NodeConfig,  
12    NullTelemetrySink,  
13    RaftApplication,  
14    RaftCluster,  
15    RaftNode,  
16    RaftShell,  
17    Role,  
18    SystemClock,
```

NodeConfig

Field	Type	Description
<code>node_id</code>	<code>str</code>	Unique node identifier
<code>peers</code>	<code>list[str]</code>	Other nodes the node can talk to
<code>election_timeout_ms</code>	<code>int</code>	Base election timeout
<code>heartbeat_interval_ms</code>	<code>int</code>	Leader heartbeat interval; default 50
<code>initial_members</code>	<code>list[str] None</code>	Explicit initial membership; defaults to node plus peers

RaftCluster

Method	Description
<code>add_node(config, paths, application, period_us=0)</code>	Add a node and register its FSMs
<code>provision_node(node_id, members, running=False, election_timeout_ms=450)</code>	Create a node during reconfiguration
<code>tick(advance_ms=10)</code>	Advance one runtime tick
<code>run(ticks, advance_ms=10)</code>	Advance several ticks
<code>leader()</code>	Return the current leader or <code>None</code>
<code>submit_to_leader(command)</code>	Submit to the current leader if one exists
<code>summaries()</code>	Return per-node status dictionaries
<code>stop_node(node_id) / start_node(node_id) / restart_node(node_id)</code>	Lifecycle helpers

Method	Description
<code>export_trace(path) /</code> <code>report_trace(path) /</code> <code>serve_trace(...)</code>	Trace helpers

RaftNode

Method / property	Description
<code>role</code>	Current <code>Role</code>
<code>submit_command(command)</code>	Enqueue a client command for this node
<code>request_membership_change(members)</code>	Request full target membership
<code>summary()</code>	Return a status dictionary
<code>last_committed_command()</code>	Return the last applied command if available
<code>stop() / start()</code>	Local node lifecycle

13. Building and Testing

Run the library tests:

```
1 uv run --project python --extra dev pytest -q
```

Run the standalone demo tests:

```
1 uv run --project python/examples/demo_app --extra dev pytest -q
```

Build the standalone demo package:

```
1 cd python/examples/demo_app
2 uv build
```

14. Troubleshooting

No leader is elected

- Ensure every node has the same initial membership.
- Use staggered election timeouts in deterministic examples.
- Advance enough simulated time with `cluster.run(...)`.
- Check whether nodes were stopped through `stop_node` or `node.stop()`.

Commands are not committed

- Submit writes to the leader, or use a layer that forwards to the leader.
- Run enough ticks after submission.
- Check `status`, `leader`, and `log NODE` in the shell.

A joining node does not participate

- In in-memory clusters, call `cluster.provision_node(...)` before requesting the membership change.
- Pass the full target member list to `request_membership_change`.
- Run until the cluster returns to stable membership.

Snapshot restore misses state

- Ensure `snapshot()` returns the complete application state.
- Ensure the returned object is JSON-serializable when using `JsonFileStorage`.
- Keep all state mutation inside `apply` and `restore_snapshot`.